



CS-Lab No. 5

RP2040 Programming II

2026-06-22

Computer Structure Class Spring 2026

Sharif University of Technology

Spring 2026o

By Dr. Saeed Bagheri Shouraki

I. Objective

The goal of this prelab is to introduce you to a few key functionalities of the RP2040 micro-controller, alarm and repeating alarm usage, calculating PWM frequencies, and generating analog signals using digital techniques. By the end of this prelab, you should have a solid understanding of how these components work individually and how they can be utilized using the micropython interpreter.

II. Concepts and Ideas

II.1. Reading a Matrix Keyboard

A matrix keyboard is a compact way to handle multiple button inputs using rows and columns. Each button is at the intersection of a specific row and column. To detect which button is pressed:

1. Set one row high at a time while keeping others low.
2. Check the columns for a high signal, which corresponds to the pressed button.

This scanning approach minimizes the number of GPIO pins required. You will implement this functionality by programming the GPIO pins on the RP2040. Using the **pull-down** or **pull-up** feature of the μC is helpful for accomplishing this task.

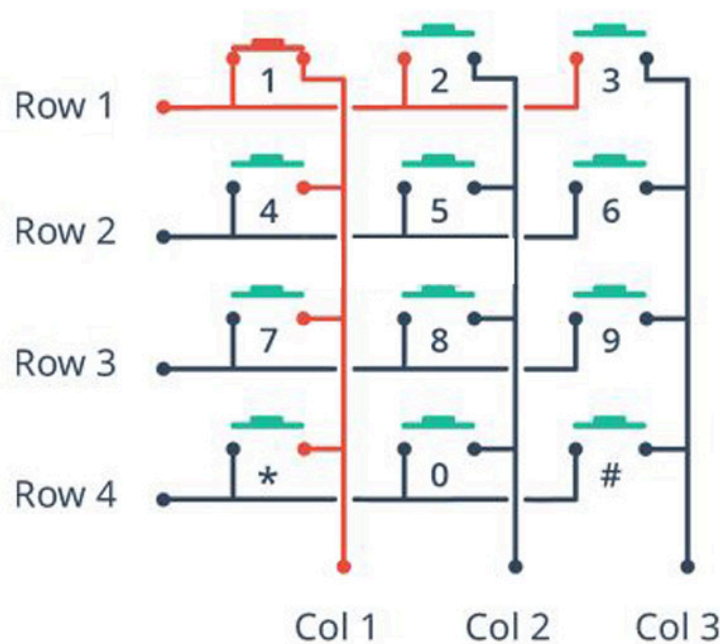


Fig. 1 — Schematic of a Matrix Keypad

II.2. Using the Timer Functionality of the RP2040

The RP2040's hardware timers allow precise scheduling of tasks. A repeating alarm lets you execute code at fixed intervals, enabling periodic operations like sampling sensors or updating a display. For instance:

- Single Alarm: Configures a one-time task using a timer.
- Repeating Alarm: Extends this to trigger tasks repeatedly at a specified interval.

You will learn to configure these alarms and timers using micropython, exploring their utility in time-critical tasks.

- See [Timer Quick Reference](#).

II.3. Finding the PWM Frequency

Pulse Width Modulation (PWM) is used to control devices like motors and LEDs or generate signals. The PWM frequency depends on:

- System Clock: The clock driving the microcontroller. The μC can have many clock signals derived from various PLLs available on the chip. In RP2040, the clock source for the PWM signal is the same as the CPU's clock.
- Clock Divider: Reduces the system clock frequency in case a lower PWM frequency is desired.
- Period: Also known as the PWM resolution or wrap target or counter target. By increasing the wrap target, the duty cycle can be adjusted with more precision at the cost of reduced output frequency.

The relationship can be expressed as:

$$f_{\text{PWM}} = \frac{f_{\text{sys_clk}}}{\text{clk_div} \times \text{wrap_target}}$$

You will calculate the appropriate frequency for your application and configure the RP2040's PWM hardware accordingly.

Note

- See [Section 4.5](#) from the RP2040 datasheet from raspberry pi foundation for a more detailed explanation.

II.4. Sinewave Generation

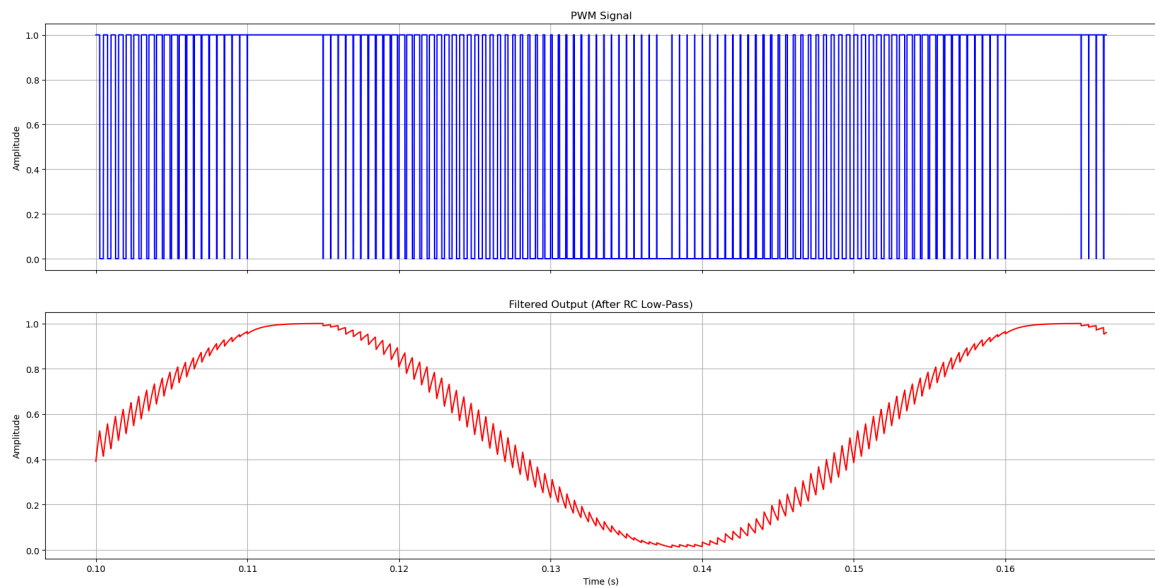


Fig. 2 — Sinewave PWM Signal

Generating analog signals like sinewaves with a digital microcontroller involves:

1. Using a repeating alarm to trigger duty cycle value updates.
2. Outputting digital values representing a sinewave using a PWM signal.
3. Filtering the PWM output through an RC lowpass filter to smooth it into a continuous sinewave.

Unlike the more common method of using a PWM wrap interrupt for updating, this approach avoids direct interrupt handling, making it simpler to implement.

- See the below code example for LED fading (does not generate sinewave PWM):

III. Tasks

Note

Directly **using interrupts** for any of the tasks below **will disqualify** your code.

1. Matrix Keyboard Scanning
 - Explain how rows and columns can be used to detect individual button presses in a matrix keyboard.
 - What challenges, such as debouncing, might arise, and how can they be addressed?
 - Explain how simultaneously pushing two buttons can short two driver pins and how this hazard can be eliminated by only using the pull-up and pull-down resistors and *not* using the GPIOs in the output direction.
 - Write code that scans (reads) a 4x3 matrix keypad and implements appropriate debouncing measures.

2. Alarm Functionality

- How does the RP2040 repeating alarm mechanism differ from a simple delay function?
- In what situations would you prefer a repeating alarm over polling?

3. PWM Configuration

- Find the default `sys_clk` frequency of raspberry pi pico using the C-SDK.
- Write code that dims the onboard LED with a dimming resolution of at least `12 bits` and a PWM frequency of at least 3 KHz.

4. Sinewave Generation

- Why is a lowpass filter necessary for converting a PWM signal into a sinewave?
- Describe the role of the repeating alarm in the sinewave generation process.
- Write code that generates a 20 Hz sine PWM, design an appropriate RC filter, and approximate the output ripple.
- Explain why increasing the PWM frequency is desirable in this situation.